Software Requirements Specification — Puong

Puong — a shared family archive.

Field	Value
Project / Document	Puong — shared family archive · Software Requirements Specification
Version / Date	1.0 · 21 October 2026
Author	Chheng Sokuntheary
Student ID	p20240044
Course	Internet Programming
Lecturer	Chun Thavorac
Standard	ISO/IEC/IEEE 29148:2018 (successor to IEEE 830-1998)

Table of contents

1. Introduction · 2. Overall description · 3. Specific requirements · 4. Traceability matrix · 5. Domain reference

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of **Puong**, a web application in which a family records the people related to it, the relations between them, and the memories and photographs attached to them. Every requirement in §3 carries an identifier, and §4 maps each requirement group to the user story it came from, the component that implements it, and the test that proves it.

1.2 Scope

A user creates one or more family trees and shares them with relatives; the members of a tree collectively build the record.

In scope: authentication and role-based access control within a tree; tree management, membership, invitations and ownership transfer; people, relations and a generation-organised tree view; memories and photographs; search, filtering and pagination; Docker Compose deployment with continuous integration.

Out of scope:

Excluded	Reason
Chat, messaging, real-time collaborative editing	Real-time infrastructure, message state, conflict handling and socket authentication are far more work than the demo needs.
Friend requests, blocking, member profiles	A friend is only another relation type on a person's page. Requests and blocking turn the archive into a social network.
Timeline view	A third layout of data the tree view and the filtered list already cover.
GEDCOM import/export	XML parsing is a project of its own and would compete with graded features.
Email delivery, password reset	Requires a mail provider and external credentials. Invitations are stored and shown in the application instead.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
Tree	A named family record owned by one user and shared with members.

Term	Meaning
Member	A user holding a `Membership` in a tree, carrying a role and a status.
Owner / Contributor / Viewer	The member who created the tree and controls its membership / a member who may add and edit their own contributions / a member who may read shared content but change nothing.
Person	A human being recorded in a tree, including relatives and acquaintances.
Relation	A typed, directed link between two people in the same tree.
Memory	A dated piece of writing, optionally with a photograph, attached to a person.
Generation	The depth of a person in the lineage, used to lay out the tree view.
Ownership	The creator of a person or a memory. An owner may override it; a contributor may not.
FR / NFR / IF	Functional / Non-functional / Interface requirement.

1.4 References

Reference	Relevance
ISO/IEC/IEEE 29148:2018	Systems and software engineering — requirement engineering; the process and document structure followed here.
RFC 7519	JSON Web Token; the session token format used for authentication.
RFC 9110	HTTP Semantics; the status codes and methods used by the API.
RFC 6749	The OAuth 2.0 Authorization Framework; referenced for its bearer-token model only.
OWASP ASVS	Application Security Verification Standard; the source of the security requirements in §3.5.

1.5 Overview

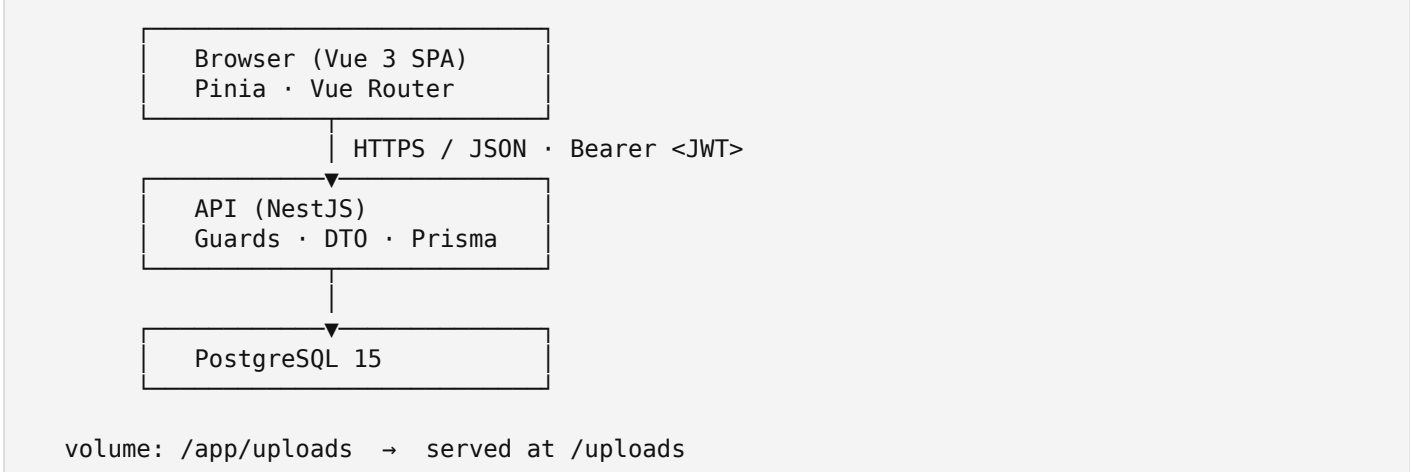
Puong has two tiers: a **web** tier (Vue 3 single-page application) and an **api** tier (NestJS REST service on PostgreSQL through Prisma). They are separate deployable units communicating only over HTTPS with JSON. §2 describes the product in the abstract; §3 is the normative part; §4 traces requirements to implementation and test.

---

2. Overall description

2.1 Product perspective

Puong is self-contained and depends on no third-party service at run time. The API is the only component that touches the database; the browser never does.



2.2 Product functions

Seven functional groups, which the ten requirement groups in §3.2 implement: **authentication and account**; **trees**; **membership and invitations** (inviting, accepting, declining, expiry, removal, ownership transfer); **people** (each person's record, and whether that record is complete); **relations** (linking two people with a type, under the rules that keep the lineage sound); **memories and photographs** (with per-memory privacy); **discovery** (search, filter, pagination, and the generation-column tree view).

2.3 User stories

The ten stories below define the scope of the release: nine are in the MVP and story 8 is nice to have. §4 traces each requirement group back to the stories it serves.

ID	Role	User story
US-1	Owner	As an Owner, I want to create a family tree and give it a name, so that we have one place for our family.
US-2	Owner	As an Owner, I want to invite a relative by email and choose their role, so that the whole family can contribute.
US-3	Member	As any member, I want to see the whole tree organised by generation, so that I can understand how everyone is connected.
US-4	Contributor	As a Contributor, I want to add a person with their name and dates, so that the record grows.
US-5	Contributor	As a Contributor, I want to link a new person to relatives already in the tree, so that I do not enter the same family twice.
US-6	Contributor	As a Contributor, I want to attach a memory and a photograph to a person, so that the family's stories and pictures are kept together.
US-7	Contributor	As a Contributor, I want to edit and delete only my own contributions, so that I can fix my mistakes without touching anyone else's.
US-8	Owner	As an Owner, I want to see which profiles are still incomplete, so that the record gets finished over time. **Nice to have — outside the MVP.**
US-9	Member	As any member, I want to search for a person and filter the list, so that I can find anyone quickly.
US-10	Owner	As an Owner, I want to nominate a successor who can take over the tree, so that the archive outlives the person who started it.

2.4 User characteristics

Role	Frequency	Expertise	What they do
Owner	Weekly	Low technical	Creates the tree, invites relatives, curates people and relations, tracks incomplete profiles, nominates a successor
Contributor	Occasionally, often from a phone	Low technical	Adds people and relations, writes memories, uploads photographs, corrects their own entries
Viewer	Rarely	Very low technical	Looks at the tree, reads shared memories, changes nothing

There is no administrator role distinct from the Owner, and no role that can read a tree it does not belong to.

2.5 Constraints

Business — a person may belong to any number of trees, but a tree has exactly one owner at a time; the last active owner cannot be removed or demoted; a memory is private to its author or shared with the tree; a person may not become their own ancestor.

Technical — TypeScript in strict mode; NestJS with Prisma on PostgreSQL 15; Vue 3 with Vite; the whole stack starts with a single `docker compose up`; the API is documented in OpenAPI 3; no external service is required at run time.

Delivery — milestones at the end of weeks 3, 5, 7, 9, 12 and 14; code frozen at the end of week 15; the defence includes a live change made to the running application.

2.6 Assumptions and dependencies

- PostgreSQL 15 is available in the development environment and as a container image.
- Browsers support ES2020, `localStorage` and the `File` API; photographs are uploaded as `multipart/form-data`.
- Identity is established solely by the signed JWT issued by the API; no session store is shared between instances.
- Time-dependent behaviour — chiefly the expiry of an invitation — is evaluated by the API from the timestamps already stored, not by the browser.

---

3. Specific requirements

"Shall" marks a mandatory requirement, **"should"** a recommended one, **"may"** an optional one. Each requirement is independent and testable; §4 names the verification.

3.1 External interface requirements

API interface

- **IF-1.1** The API shall expose a JSON over HTTP interface under the base path `/api`, and shall authenticate callers by reading a bearer token from the `Authorization` header.
- **IF-1.2** The API shall return every error in the single shape `{ "statusCode": <int>, "message": <string | string[]>, "error": <string> }`.
- **IF-1.3** The API shall publish its OpenAPI 3 documentation at `/api/docs`.
- **IF-1.4** The API shall accept and return `application/json` on every endpoint except photograph upload, which shall accept `multipart/form-data`.
- **IF-1.5** The API shall use resource-based URLs, one resource per path segment.
- **IF-1.6** The API shall return `200` for a successful read or update, `201` for a successful creation with a body, `204` for a successful deletion, and the error codes `400`, `401`, `403`, `404` and `409` as specified elsewhere in this document.
- **IF-1.7** The API shall reject a request body that fails validation with `400 Bad Request` and a message naming the offending fields, and shall never include a stack trace in an error response.

User interface

- **IF-2.1** The web tier shall be a Vue 3 single-page application written in TypeScript with strict mode enabled, using Vue Router for navigation and Pinia for shared state.
- **IF-2.2** The web tier shall guard every route that requires authentication, and shall keep view-local state inside components.
- **IF-2.3** The web tier shall hide any control whose action the current member's role does not permit.
- **IF-2.4** The web tier shall validate all forms on the client and shall display the field-level error returned by the API.
- **IF-2.5** The web tier shall present a loading state, an empty state and an error state for every list view.

3.2 Functional requirements

FR-1 Authentication and account

- **FR-1.1** The system shall register a user with an email address and a password.

- **FR-1.2** The system shall reject a registration whose email is already registered or is not a valid address format.
- **FR-1.3** The system shall hash a password with bcrypt at a cost factor of at least 10 before storing it, and shall never store or log a password in plain text.
- **FR-1.4** The system shall authenticate a registered user against the stored hash and shall issue a signed JWT on success.
- **FR-1.5** The system shall reject authentication with a wrong password with the same message as for an unknown email, so that registered addresses cannot be discovered.
- **FR-1.6** The system shall return `401 Unauthorized` for a protected endpoint when the token is missing, malformed or expired, and shall allow a user to log out and discard the client token.

FR-2 Trees

- **FR-2.1** The system shall allow an authenticated user to create a tree with a name, and shall assign the role `OWNER` to the creator.
- **FR-2.2** The system shall reject a tree name that is empty, shorter than 2 characters or longer than 120 characters.
- **FR-2.3** The system shall list only the trees in which the calling user holds an active membership.
- **FR-2.4** The system shall allow an owner to rename a tree, change its description, set a cover photograph, and delete the tree together with its people, relations, memories and memberships.
- **FR-2.5** The system shall return `403 Forbidden` when a non-member requests a tree, and `404 Not Found` when the caller is a member but the tree does not exist.

FR-3 Membership, invitations and roles

- **FR-3.1** The system shall allow an owner to invite an existing user by email with a role of `CONTRIBUTOR` or `VIEWER`, and shall reject an invitation to an address that has no account.
- **FR-3.2** The system shall store an invitation with the status `INVITED` and shall display pending invitations on the invited user's home page.
- **FR-3.3** The system shall allow an invited user to accept or decline an invitation, setting the membership status to `ACTIVE` or `REVOKED` accordingly.
- **FR-3.4** The system shall expire an invitation that has not been accepted within 7 days of being sent.
- **FR-3.5** The system shall allow an owner to change the role of a member or to remove a member, and shall prevent the removal or demotion of the last active owner of a tree.
- **FR-3.6** The system shall return `403 Forbidden` when a contributor or a viewer attempts any action reserved to an owner.

FR-4 People

- **FR-4.1** The system shall allow a contributor to create a person within a tree they belong to, requiring only the full name and accepting optional birth date, death date, gender, nickname, biography and photograph.
- **FR-4.2** The system shall reject a death date earlier than the birth date of the same person.
- **FR-4.3** The system shall derive a person's profile completeness from the person's own data instead of storing it: a profile counts as complete when the person has a full name, a birth date, and either a photograph or a biography. No status column shall exist on `Person`, so editing a person cannot leave a stale status behind.
- **FR-4.4** The system shall allow a contributor to update a person they created, and shall allow an owner to update or delete any person in the tree.
- **FR-4.5** The system shall return `403 Forbidden` when a contributor attempts to update a person created by another member.
- **FR-4.6** The system shall allow an owner to delete a person, and shall remove that person's relations and memories with it, so that no orphaned record is left behind.

- **FR-4.7** *(nice to have — serves story 8, outside the MVP)* The system shall show an owner the number of people in the tree whose derived profile completeness is incomplete.

FR-5 Relations

- **FR-5.1** The system shall allow a contributor to create a relation between two different people in the same tree, with a type of `PARENT`, `SPOUSE`, `FRIEND` or `SIBLING`, and shall reject a relation whose source, target and type already exist.
- **FR-5.2** The system shall interpret a `PARENT` relation as meaning that the source person is the parent of the target person.
- **FR-5.3** The system shall reject a relation between a person and themselves with `400 Bad Request`.
- **FR-5.4** The system shall reject a `PARENT` relation that would make a person an ancestor of themselves, and shall return `409 Conflict` with a message explaining the cycle.
- **FR-5.5** The system shall allow a contributor to delete a relation. A relation carries no author, so any member with write access may create or delete any relation in the tree; the unique triple (`fromId`, `toId`, `type`) prevents the same relation being created twice.
- **FR-5.6** The system shall place a person who has no `PARENT` relation in generation 0, and any other person one generation below the **highest** generation among their parents, so that every parent is in an earlier generation than their child. A person who has no `PARENT` relation shall then adopt the highest generation among the people they are connected to by `SPOUSE`, `SIBLING` or `FRIEND`, repeating until no generation changes; a person with no `PARENT` relation and no connection to a placed person shall remain in generation 0.
- **FR-5.7** The system shall include in a person's detail response their parents, children, spouses and other relations.

FR-6 Memories and photographs

- **FR-6.1** The system shall allow a contributor to attach a memory to a person in a tree they belong to, requiring a title and a body and accepting an optional occurred date and an optional photograph.
- **FR-6.2** The system shall accept a photograph — whether attached to a person or to a memory — only in JPEG, PNG or WebP format and only up to 5 MB, shall reject any other type or size with a clear message, and shall store the file on local disk under a directory mounted as a Docker volume.
- **FR-6.3** The system shall serve stored photographs over HTTP from a dedicated route.
- **FR-6.4** The system shall allow a contributor to edit and to delete their own memories.
- **FR-6.5** The system shall return `403 Forbidden` when a contributor attempts to edit or delete a memory authored by another member.
- **FR-6.6** The system shall show a memory with visibility `PRIVATE` only to its author, including to an owner, and shall show a memory with visibility `TREE` to every active member of the tree.
- **FR-6.7** The system shall list the memories of a person in reverse chronological order of their occurred date, and shall not allow the recorded author of a memory to be changed afterwards.
- **FR-6.8** The system shall allow an owner to delete a memory authored by another member only when that memory's visibility is `TREE`, and shall return `403 Forbidden` when anyone other than the author attempts to delete a memory marked `PRIVATE`. No member shall be allowed to edit a memory they did not author.

FR-7 Search, filter and pagination

- **FR-7.1** The system shall return the people of a tree whose full name contains a given search term, compared without regard to case.
- **FR-7.2** The system shall filter people by relation type, by living status, and by generation, and shall restrict every search and filter result to the tree named in the request.
- **FR-7.3** The system shall paginate the people list with a default page size of 10 and a maximum page size of 50, and shall return the total number of matching people and the total number of pages alongside the results.

- **FR-7.4** The system shall reject a request whose page, page size or filter value is invalid with `400 Bad Request`.

FR-8 Tree view

- **FR-8.1** The web tier shall present the people of a tree grouped into columns, one column per generation, showing each person exactly once.
- **FR-8.2** The web tier shall allow a member to select a person and shall show that person's relations.
- **FR-8.3** The web tier shall highlight the ancestor path and the descendant path of the selected person.
- **FR-8.4** The web tier shall show an explicit empty state when a tree contains no people.

FR-9 Transfer of ownership

- **FR-9.1** The system shall allow an owner to nominate one active member whose role is `CONTRIBUTOR` or `OWNER` as successor, and shall present the pending nomination to that successor.
- **FR-9.2** The system shall allow the successor to accept or decline the nomination; on acceptance ownership shall move to the successor and the previous owner shall be demoted to `CONTRIBUTOR`.
- **FR-9.3** The system shall record each transfer with its date and the identity of the previous owner.

FR-10 Scheduled work *(nice to have — outside the MVP)*

The MVP has no background job: invitation expiry (FR-3.4) is evaluated when an invitation is read. The requirement below would only write that state to the database.

- **FR-10.1** The system shall run a daily job inside the API process that sets any invitation older than 7 days to the status `REVOKED`.

3.3 Internal requirements

- **IR-1** The system shall store its data in PostgreSQL 15 and shall access it only through Prisma migrations.
- **IR-2** The schema shall consist of the tables `User`, `Tree`, `Membership`, `Person`, `Relation` and `Memory`, and shall be created and populated by a seed script that is safe to run repeatedly.
- **IR-3** The `Membership` table shall hold the unique pair (`treeId`, `userId`); the `Relation` table shall hold the unique triple (`fromId`, `toId`, `type`).
- **IR-4** A person shall reference exactly one tree; a memory shall reference exactly one person and exactly one author.
- **IR-5** Deleting a tree shall cascade to its memberships, people, relations and memories; deleting a user shall be restricted, so that a tree must first be transferred or deleted.

Data model. The six tables below are the whole persisted model. The cardinality of each relation is stated in the paragraph that follows the table.

Entity	Key attributes	Relation
`User`	id, email (unique), passwordHash, displayName	owns `Trees`; holds `Memberships`; authors `Memorys`
`Tree`	id, name, description, coverPhotoUrl, ownerId, successorId	one owner `User`; many `Memberships`, `Persons`, `Relations`, `Memorys`
`Membership`	id, treeId, userId, role, status, invitedAt, acceptedAt	joins `User` and `Tree`; unique per (tree, user)
`Person`	id, treeId, fullName, nickname, gender, birthDate, deathDate, isLiving, photoUrl, bio, createdById	one `Tree`; one creator `User`; many `Relations`; many `Memorys`
`Relation`	id, treeId, fromId, toId, type	connects two `Persons`; unique per (from, to, type)
`Memory`	id, treeId, personId, authorId, title, body, happenedAt, photoUrl, visibility	one `Person`; one author `User`; one `Tree`

`Tree → Person` and `Person → Memory` are one-to-many, implemented with a foreign key. `User ↔ Tree` is many-to-many through `Membership`, which carries the role and the invitation status so permission is decided in one place; `Person ↔ Person` is many-to-many through `Relation`, whose `type` column covers parent, spouse, friend and sibling in one table.

3.4 Software system constraints

- **CON-1** The implementation shall be TypeScript with `strict: true` in both tiers.
- **CON-2** The API shall be built with NestJS and the data layer with Prisma; the web tier with Vue 3, Vite, Vue Router and Pinia.
- **CON-3** The backend shall be organised into one module per resource, each with a controller, a service and its DTOs.
- **CON-4** The system shall be delivered with a Docker Compose file that starts the web tier, the API, the database and the upload volume together.
- **CON-5** The repository shall contain no committed secret but a `.env.example` with placeholder values, shall contain an ESLint configuration that passes with no errors, and shall be public or private with the lecturer added as a collaborator.
- **CON-6** Commits shall be made at least weekly, with messages that describe the change.

3.5 Software quality attributes

Performance

- **NFR-1.1** The API shall return the result of a people search within 500 ms for a tree of up to 500 people.
- **NFR-1.2** The web tier shall render the tree view of 500 people within 2 seconds.

Security

- **NFR-2.1** The system shall hash every password with bcrypt at a cost factor of at least 10, and shall never store or log a password in plain text.
- **NFR-2.2** The system shall verify the signature and the expiry of every bearer token before acting on it.
- **NFR-2.3** The system shall verify the calling member's role in the target tree and the ownership of the target record before every mutating action, and shall return `403 Forbidden` otherwise.
- **NFR-2.4** The system shall read all secrets from environment variables only, and shall keep `.env` out of version control.
- **NFR-2.5** The system shall restrict CORS to the configured web origin.
- **NFR-2.6** The system shall parameterise every database query through the ORM and shall never concatenate user input into SQL, and shall never render user-supplied text as raw HTML.

Reliability and maintainability

- **NFR-3.1** The core domain rules — ancestor cycle detection, generation computation, profile completeness, permission decisions and memory visibility — shall be implemented as unit-testable functions in the service layer.
- **NFR-3.2** The API shall be covered by end-to-end tests for the main endpoints, including at least one test asserting `403 Forbidden` for each restricted action.
- **NFR-3.3** The codebase shall pass ESLint with zero errors and shall compile with TypeScript strict mode.

Portability and deployment

- **NFR-4.1** The whole stack shall start with a single `docker compose up` command, and the API shall run behind a reverse proxy that terminates HTTPS in the deployed environment.
- **NFR-4.2** The CI workflow shall run lint, unit tests and a build on every push, and shall fail the build when any of them fails.

Usability

- **NFR-5.1** Every list view in the web tier shall provide a loading state, an empty state and an error state.

---

4. Traceability matrix

Each row groups the requirements of one functional area and states the user story it originates from, the component that implements it, and the test that proves it. "Story" refers to the user stories in §2.3. A requirement is *verified* only when the named test exists and passes in CI.

Requirements	Story	Implementation	Verification	Status
FR-1.1 – FR-1.6	—	`AuthModule`, `auth.service`, `JwtStrategy`	`auth.service.spec`, `auth.e2e-spec`	planned
FR-2.1 – FR-2.5	US-1	`TreesModule`, `trees.controller`	`trees.service.spec`, `trees.e2e-spec`	planned
FR-3.1 – FR-3.6	US-2, US-10	`MembershipsModule`, `TreeMembershipGuard`	`memberships.e2e-spec`	planned
FR-4.1 – FR-4.6	US-4	`PersonsModule`, `persons.service`	`persons.service.spec`, `persons.e2e-spec`	planned
FR-4.7 *(nice to have)*	US-8 *(nice to have)*	`PersonsService.countIncomplete`	`persons.service.spec`	optional
FR-5.1 – FR-5.7	US-5	`RelationsModule`, `RelationService.assertNoAncestorCycle`	`relations.service.spec`	planned
FR-6.1 – FR-6.8	US-6, US-7	`MemoriesModule`, `UploadInterceptor`	`memories.e2e-spec`	planned
FR-7.1 – FR-7.4	US-9	`PersonsService.findAll` query builder	`person-query.spec`, `persons.e2e-spec`	planned
FR-8.1 – FR-8.4	US-3	`TreeView.vue`, `useTreeLayout` composable	`tree-layout.spec`	planned
FR-9.1 – FR-9.3	US-10	`TreesService.transferOwnership`	`trees.service.spec`	planned
FR-10.1 *(nice to have)*	US-2	`ScheduledTasksService`, `@Cron`	`scheduled-tasks.spec`	optional
IF-1.1 – IF-1.7	—	Global `HttpExceptionFilter`, `ValidationPipe`	`error-format.spec`	planned

The status column is updated at each milestone and is the basis of the features-versus-requirements assessment in the final product grade.

---

5. Domain reference

5.1 Test accounts

The application ships with three seeded accounts, one per role, so that the access rules of §3.2 can be exercised without setup.

Role	Email	Password	Access
Owner	`owner@puong.dev`	`Passw0rd!`	Owner of the Nguyễn tree and the Trần tree
Contributor	`contributor@puong.dev`	`Passw0rd!`	Contributor in the Nguyễn tree; not a member of the Trần tree

Role	Email	Password	Access
Viewer	`viewer@puong.dev`	`Passw0rd!`	Viewer in the Nguyễn tree; invited but not accepted in the Trần tree

The seed also creates a four-generation tree of about 22 people with shared and private memories and several deliberately incomplete profiles, so that the tree view, the filters and the "needs details" count are meaningful on a fresh start.

5.2 Glossary of domain rules

1. **Generation** — one below the highest generation among a person's parents. A person with no parents recorded starts at 0 and then adopts the highest generation of the people they are linked to, remaining 0 only when linked to nobody placed.
2. **Ancestor cycle** — a proposed `PARENT` relation that would place a person above themselves in the lineage. Rejected with `409`.
3. **Profile completeness** — derived from the person's data and never stored: a profile is complete when the person has a full name, a birth date, and either a photograph or a biography.
4. **Visibility** — `PRIVATE`, shown to the author alone, or `TREE`, shown to every active member.
5. **Successor** — the member nominated by an owner to take over the tree on acceptance.